

Corporación Unificada Nacional de Educación Superior (CUN)

**Implementación de una Red Domótica con WiFi en Cisco Packet
Tracer:
Automatización mediante Conditions y Programación en Java
(Spring Boot)**

Arlez Camilo Ceron Herrera

Facultad de Ingeniería

Redes Inalámbricas (54496) — Primer Bloque, Grupo 26V04

Docente: Ricardo Alfredo López Bulla

Septiembre de 2026

Tabla de contenido

1. Introducción y objetivos	3
Objetivo general	3
Objetivos específicos	3
2. Historias de usuario	4
3. Topología de red	4
3.1 Direccionamiento IP	5
4. Configuración de la red WiFi	5
5. Administración desde el dispositivo móvil	7
6. Automatización con Conditions	7
7. Programación en Java (Spring Boot)	8
7.1 Arquitectura del backend	8
7.2 Endpoints principales	8
7.3 Lógica de automatización (extracto)	9
7.4 Documentación y pruebas	9
8. Pruebas y evidencias	10
9. Control de versiones (GitHub)	11
Estructura final del repositorio	11
10. Conclusiones	11
Referencias	12

1. Introducción y objetivos

Este informe documenta el diseño e implementación de una red domótica simulada en Cisco Packet Tracer, conectada mediante una red inalámbrica WiFi, que integra dispositivos inteligentes (ventana, puerta, lámpara y aire acondicionado, entre otros) administrables desde un dispositivo móvil. Adicionalmente, se implementó automatización mediante reglas condicionales dentro del simulador y un backend en Java (Spring Boot) que replica la misma lógica de negocio, documentado con Swagger/OpenAPI. El proyecto se desarrolla en el marco del curso Redes Inalámbricas de la Corporación Unificada Nacional de Educación Superior (CUN).

Objetivo general

Diseñar, implementar y probar una red domótica funcional en Packet Tracer, administrable de forma remota desde un dispositivo inalámbrico, incorporando automatización y un componente de programación en Java.

Objetivos específicos

- Diseñar una topología de red WiFi doméstica con un Home Gateway como controlador central.
- Integrar dispositivos IoT (puerta, ventana, lámpara, aire acondicionado, sensores de movimiento y temperatura).
- Configurar el control remoto de los dispositivos desde un smartphone simulado.
- Implementar automatización basada en condiciones (temperatura y movimiento).
- Desarrollar un backend en Java/Spring Boot que replique la lógica de automatización, documentado con Swagger.
- Validar el funcionamiento mediante pruebas funcionales y documentar la evidencia correspondiente.

2. Historias de usuario

Se definieron seis historias de usuario que guiaron el diseño funcional del sistema y sirvieron de base para las reglas de automatización y los endpoints del backend Java. Cada una fue documentada como Issue en el repositorio de GitHub del proyecto.

ID	Historia de usuario	Prioridad
HU1	Como usuario, quiero encender/apagar la lámpara desde mi celular, para controlar la iluminación remotamente.	Alta
HU2	Como usuario, quiero que el A/C se encienda automáticamente si la temperatura supera 28°C, para mantener el confort sin intervención manual.	Alta
HU3	Como usuario, quiero ver el estado de la puerta principal desde el celular, para verificar la seguridad del hogar.	Media
HU4	Como usuario, quiero que la lámpara se encienda automáticamente si se detecta movimiento, para mejorar la seguridad nocturna.	Media
HU5	Como usuario, quiero abrir/cerrar la ventana desde mi celular, para controlar la ventilación remotamente.	Media
HU6	Como usuario, quiero un modo "Away" que apague lámpara y A/C con un solo control, para ahorrar energía al salir de casa.	Baja

3. Topología de red

La red se compone de un Home Gateway que actúa como punto de acceso WiFi y controlador domótico central, un smartphone que se conecta a la red para administrar los dispositivos, y seis dispositivos IoT conectados de forma inalámbrica: puerta, ventana, lámpara, aire acondicionado (Ceiling Fan), sensor de movimiento y sensor de temperatura. Todas las conexiones se establecieron mediante WiFi (líneas punteadas en el diagrama), sin cableado físico.

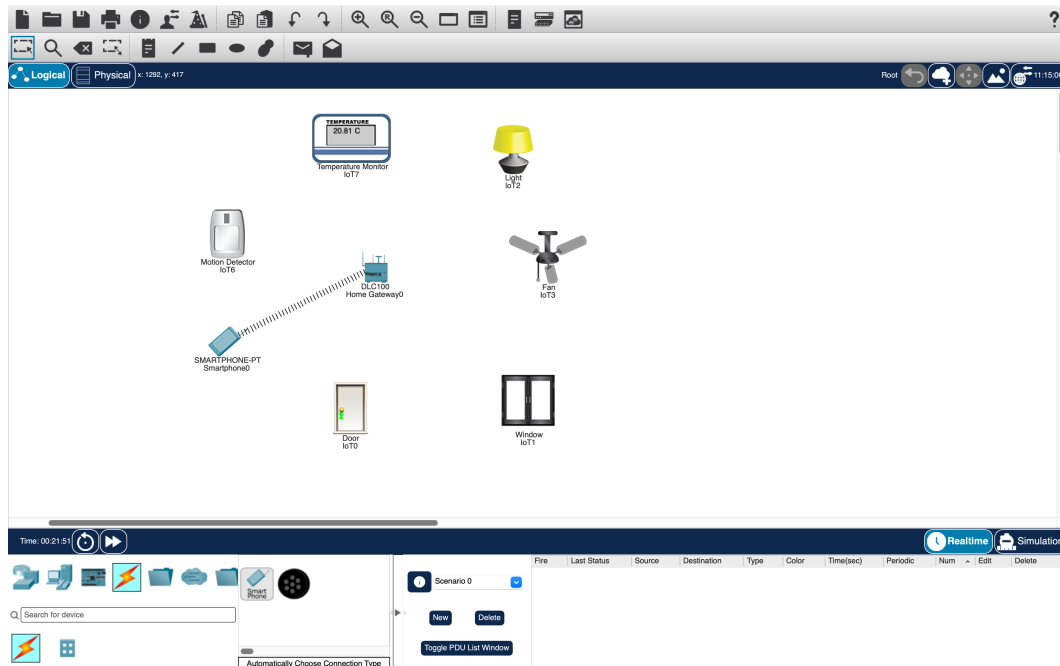


Figura 1. Topología completa de la red domótica en Cisco Packet Tracer.

3.1 Direcccionamiento IP

Dispositivo	Interfaz	Configuración IP
Home Gateway (DLC100)	LAN / Wireless	192.168.25.1 / 255.255.255.0 (DHCP Server)
Smartphone-PT	Wireless0	DHCP — 192.168.25.109 / 255.255.255.0
Puerta (IoT0)	Wireless0	DHCP
Ventana (IoT1)	Wireless0	DHCP
Lámpara (IoT2)	Wireless0	DHCP
Ceiling Fan / A/C (IoT3)	Wireless0	DHCP
Sensor de Movimiento (IoT6)	Wireless0	DHCP
Sensor de Temperatura (IoT7)	Wireless0	DHCP

4. Configuración de la red WiFi

En el Home Gateway se configuró una red inalámbrica con SSID **CasaInteligente**, autenticación WPA2-PSK y cifrado AES, replicando las condiciones de seguridad de una red doméstica real.

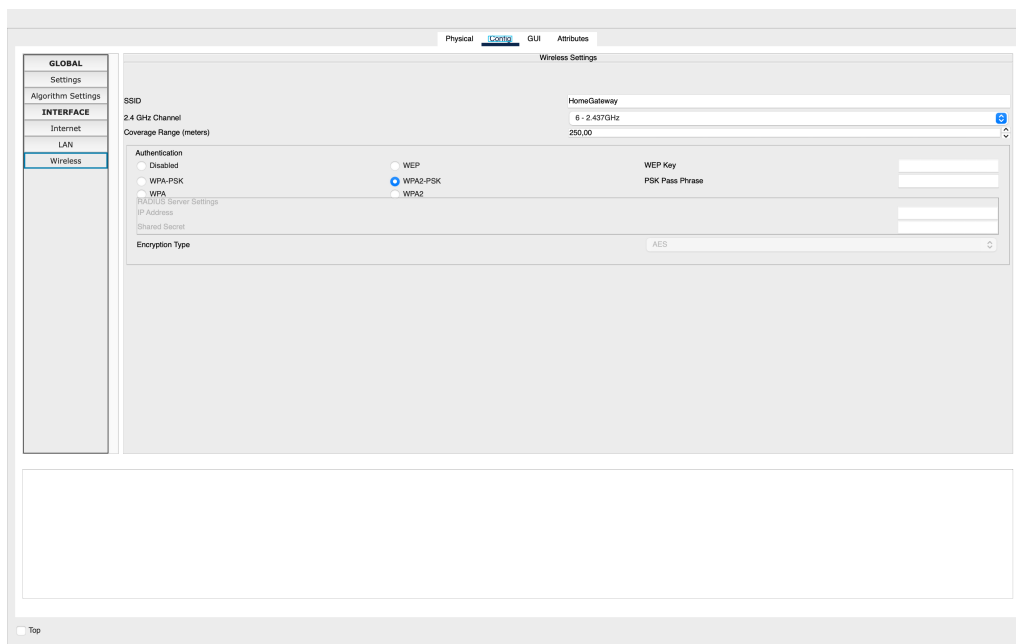


Figura 2. Configuración SSID y seguridad WPA2-PSK en el Home Gateway.

Cada dispositivo IoT y el smartphone se configuraron individualmente para unirse a esta red, ingresando el mismo SSID y contraseña. La figura 3 muestra al smartphone conectado exitosamente, con una dirección IP asignada por DHCP.

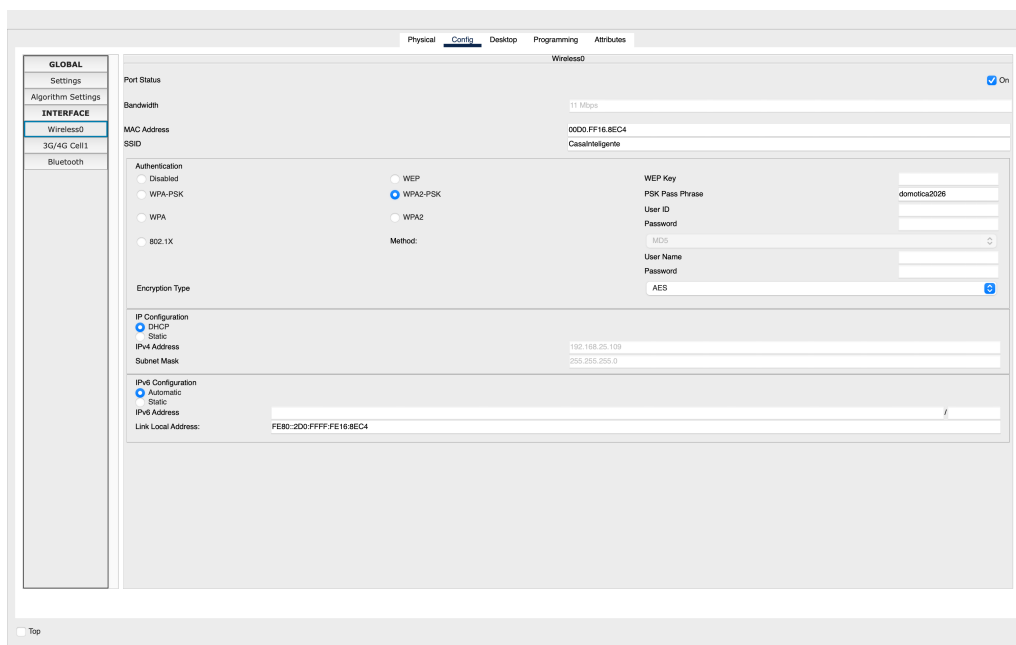


Figura 3. Smartphone conectado a la red CasaInteligente (IP 192.168.25.109).

5. Administración desde el dispositivo móvil

El Home Gateway aloja un servidor IoT accesible vía navegador web (<http://192.168.25.1>). Cada dispositivo se configuró con la opción IoT Server → Home Gateway para registrarse automáticamente en el panel central. Desde el navegador del smartphone es posible visualizar el estado de todos los dispositivos en tiempo real.

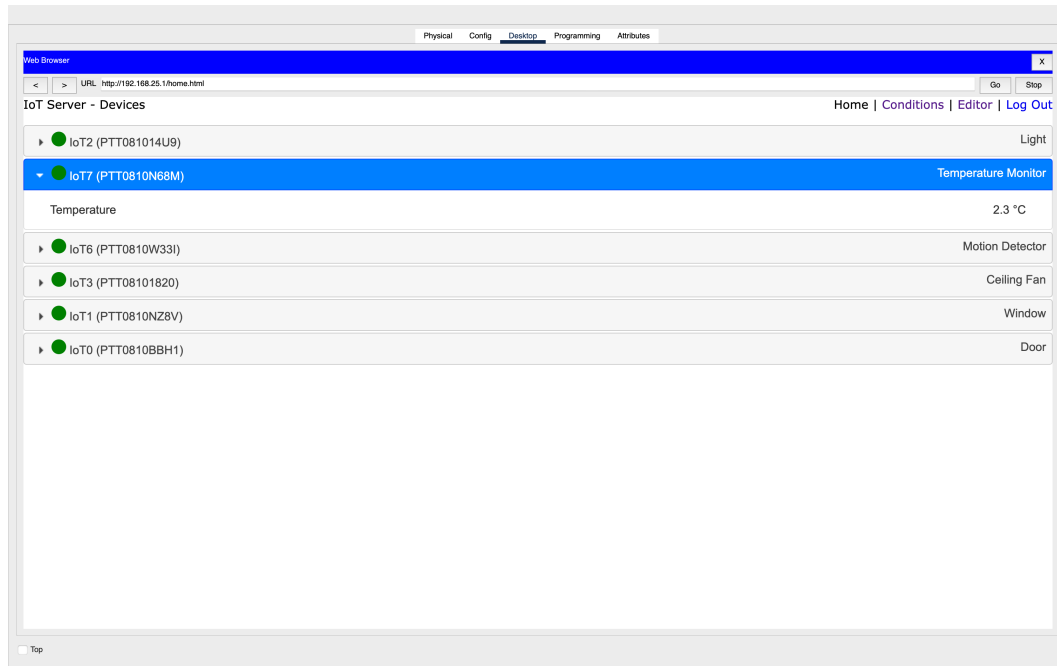


Figura 4. Panel de dispositivos IoT registrados y administrables desde el celular.

6. Automatización con Conditions

Se configuraron reglas de automatización sin código, directamente en el servidor IoT del Home Gateway, cubriendo las historias de usuario HU2 y HU4:

- HU2 — AC automático por temperatura: SI IoT7 (Temperature Monitor) > 28.0°C ENTONCES fijar IoT3 (Ceiling Fan) Status = High.
- HU4 — Luz automática por movimiento: SI IoT6 (Motion Detector) On = true ENTONCES fijar IoT2 (Light) Status = On.

Ambas reglas se crearon desde el panel web del Home Gateway (<http://192.168.25.1/conditions.html>), especificando la condición (dispositivo, atributo, operador, valor) y la acción resultante sobre el dispositivo actuador. Su funcionamiento fue validado en la sección 8 (Pruebas y evidencias).

7. Programación en Java (Spring Boot)

Cisco Packet Tracer no permite ejecutar código Java de forma nativa dentro del simulador; su motor de scripting solo admite Python, JavaScript o Blockly. Por esta razón, se desarrolló un backend independiente en Java 17 con Spring Boot 3.3.4 que replica fielmente la misma lógica de automatización configurada en Packet Tracer, representando el "cerebro" conceptual que respaldaría a la aplicación móvil.

7.1 Arquitectura del backend

El proyecto sigue una arquitectura en capas: modelos de dominio (Device, Light, AirConditioner, Door, Window, MotionSensor, TemperatureSensor), un servicio de automatización (AutomationService) y un controlador REST (DeviceController) documentado con springdoc-openapi (Swagger UI).

7.2 Endpoints principales

Método	Endpoint	Historia de usuario
GET	/api/devices/status	Estado general
POST	/api/devices/light/{on off}	HU1
POST	/api/devices/temperature/{value}	HU2
GET / POST	/api/devices/door	HU3
POST	/api/devices/motion/{true false}	HU4
POST	/api/devices/window/{open close}	HU5
POST	/api/devices/away-mode	HU6

7.3 Lógica de automatización (extracto)

```
@Service
public class AutomationService {
    private static final double TEMPERATURE_THRESHOLD = 28.0;

    public void evaluateTemperature(TemperatureSensor sensor, AirConditioner ac) {
        if (sensor.getCurrentTemperature() > TEMPERATURE_THRESHOLD) {
            ac.turnOn();
        } else {
            ac.turnOff();
        }
    }

    public void evaluateMotion(MotionSensor sensor, Light light) {
        if (sensor.isMotionDetected()) {
            light.turnOn();
        }
    }

    public void activateAwayMode(Light light, AirConditioner ac) {
        light.turnOff();
        ac.turnOff();
    }
}
```

7.4 Documentación y pruebas

El proyecto se ejecutó localmente con Maven (`mvn spring-boot:run`) y también desde IntelliJ IDEA, iniciando correctamente en el puerto 8080 (log: "Started DomoticaApplication"). La documentación interactiva de la API se generó automáticamente con Swagger UI en <http://localhost:8080/swagger-ui/index.html>. Se construyó además una colección de Postman con las 13 peticiones correspondientes a las seis historias de usuario, incluida en el repositorio (`java/red-domotica-postman-collection.json`), con la que se verificó que cada endpoint responde correctamente y que la lógica de automatización (temperatura → A/C, movimiento → lámpara, modo Away) se comporta igual que en Packet Tracer.

8. Pruebas y evidencias

Se realizaron pruebas funcionales en modo Realtime de Packet Tracer, forzando un aumento de la temperatura ambiente mediante el editor de entornos (Environments → Temperature → Ambient Temperature). Al superar el umbral de 28°C configurado en la regla HU2, el Ceiling Fan (A/C) cambió automáticamente su estado a "High", confirmando el correcto funcionamiento de la automatización end-to-end: sensor → regla → actuador.

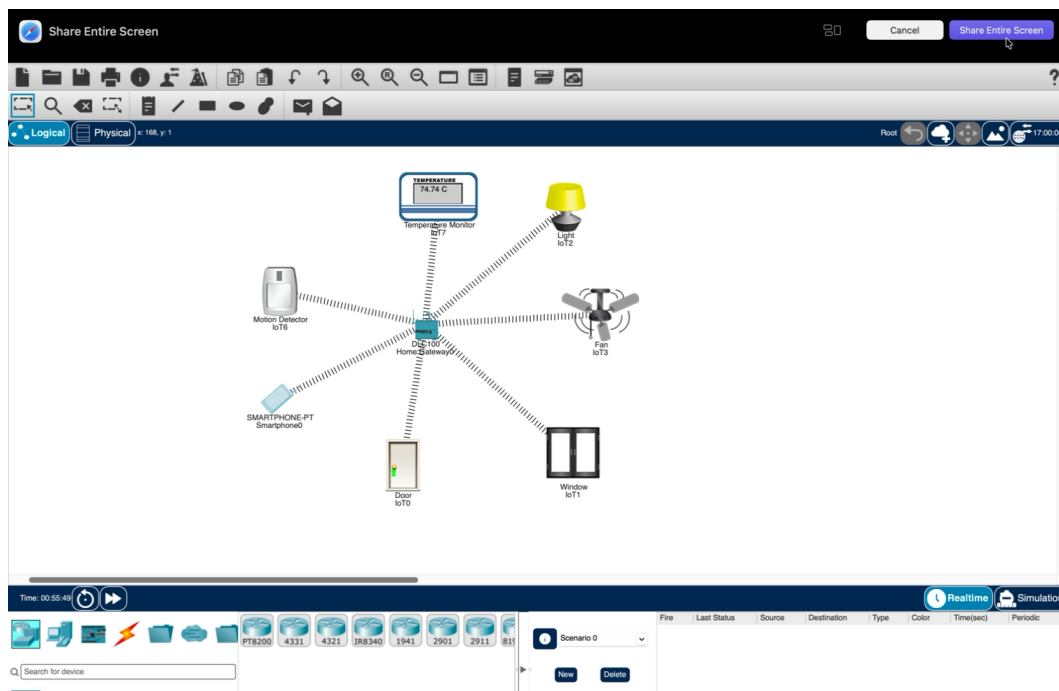


Figura 5. Evidencia de automatización: Temperature Monitor a 74.74°C y Ceiling Fan girando (encendido) de forma automática.

Adicionalmente, la lógica equivalente fue verificada exitosamente en el backend Java mediante Postman, confirmando para cada endpoint la respuesta JSON esperada (por ejemplo, al enviar temperatura mayor a 28 vía POST /api/devices/temperature/30, el campo AirConditioner de la respuesta cambia a "ON"), y en el panel de dispositivos del Home Gateway, donde los seis dispositivos IoT se muestran registrados y activos (indicador verde).

9. Control de versiones (GitHub)

Todo el proyecto se gestionó bajo control de versiones en GitHub, incluyendo el archivo .pkt, las historias de usuario, el backend Java y este informe. Se utilizó un flujo de trabajo con ramas por funcionalidad y Pull Requests hacia la rama principal.

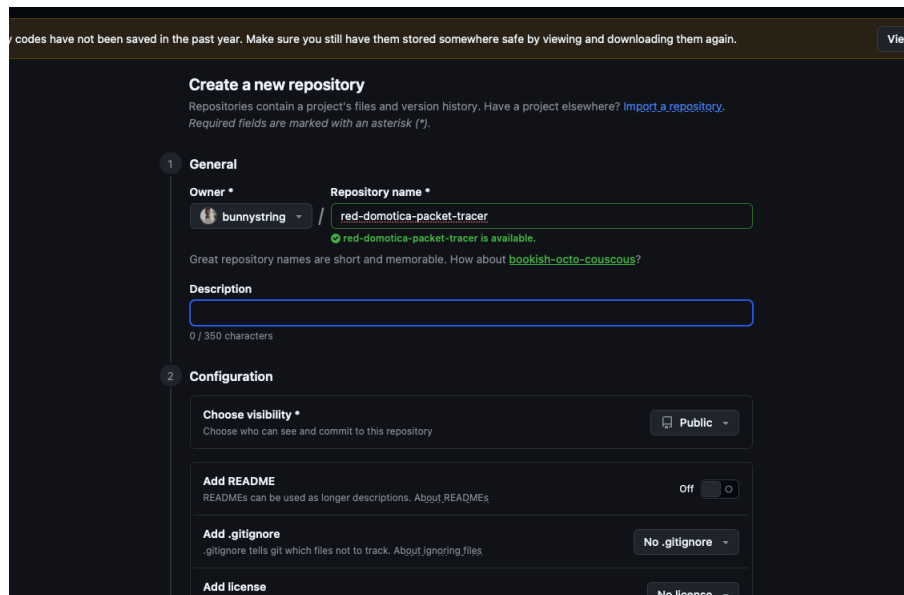


Figura 6. Creación del repositorio red-domotica-packet-tracer en GitHub.

Estructura final del repositorio

```
red-domotica-packet-tracer/  
|-- README.md  
|-- pkt/  
|   |-- red-domotica.pkt  
|-- java/  
|   |-- domotica-java/          (proyecto Spring Boot)  
|   |-- red-domotica-postman-collection.json  
|-- informe/  
|   |-- informe-tecnico-red-domotica.pdf  
-- docs/  
   |-- historias-usuario.md
```

10. Conclusiones

- Se implementó exitosamente una red domótica funcional en Cisco Packet Tracer, con seis dispositivos IoT conectados de forma inalámbrica a un Home Gateway central.

- El control remoto desde un dispositivo móvil (smartphone) se logró mediante conexión a la red WiFi CasaInteligente y acceso al panel web del Home Gateway.
- La automatización se implementó en dos niveles: reglas Conditions dentro de Packet Tracer y un backend en Java/Spring Boot que replica la misma lógica de negocio, documentado con Swagger y validado con Postman.
- Se comprobó de forma práctica que la regla de temperatura (HU2) responde correctamente ante cambios ambientales, activando el actuador correspondiente sin intervención manual.
- El proyecto quedó documentado y versionado en GitHub, incluyendo historias de usuario, código fuente y evidencia de pruebas, facilitando su trazabilidad y mantenimiento futuro.

Referencias

Cisco Systems, Inc. (2026). *Cisco Packet Tracer* (Versión 9.0.1) [Software]. Cisco Networking Academy. <https://www.netacad.com>

Pivotal Software / VMware. (2024). *Spring Boot Reference Documentation* (Versión 3.3.4). <https://docs.spring.io/spring-boot/>

SmartBear Software. (2024). *OpenAPI Specification y Swagger UI*. <https://swagger.io>

The Apache Software Foundation. (2024). *Apache Maven Project*. <https://maven.apache.org>

Postman, Inc. (2024). *Postman API Platform*. <https://www.postman.com>